

P2268R0

Freestanding Roadmap

Published Proposal, 2020-12-10

This version:

<https://wg21.link/P2268R0>

Author:

[Ben Craig](#) (NI)

Audience:

SG14

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/freestanding_proposal/blob/master/roadmap.bs

Abstract

Describe future, high level freestanding papers

Table of Contents

- 1 **Introduction**
- 2 **[P0829] pieces**
- 2.1 `<cstdlib>`, `<charconv>`, `<cmath>`, `<cinttypes>`, `<cstring>`, `<cwchar>`, and `char_traits`
- 2.2 [diagnostics], `<algorithm>`, `<numeric>`, `lock_guard`, `unique_lock`, `<span>`
- 2.3 Partial classes
- 2.4 `<random>`
- 2.5 `<chrono>` / [time]

3 Beyond [P0829]

3.1 `constexpr` as `constexpr`

3.2 Startup and termination control

3.2.1 Replaceable `std::terminate` `{#replaceable_terminate}`

3.2.2 Link-time arrays `{#link_time_arrays}`

3.2.3 Exposing existing global constructor link-time arrays `{#expose_global_ctors}`

3.2.4 `thread_local` customization points `{#thread_local_customization}`

3.3 Error handling

3.3.1 Destructor call reason `{#dtor_call_reason}`

3.3.2 Non-local exception objects `{#non_local_exception}`

3.3.3 `noexcept` blocks `{#noexcept_blocks}`

3.3.4 Trim exception header `{#trim_exception_header}`

3.4 No known path forward

3.4.1 Locked atomics `{#locked_atomics}`

3.4.2 Thread-safe statics `{#thread_safe_statics}`

3.4.3 RTTI `{#rtti}`

3.4.4 Floating point `{#floating_point}`

4 Work in progress

4.1 [P2013] Freestanding Language: Optional `::operator new`

4.2 [P1642] Freestanding Library: Easy [utilities], [ranges], and [iterators]

4.3 [P2198] Freestanding Feature-Test Macros and Implementation-Defined Extensions

5 Related work in progress

5.1 [P0581] Standard Library Modules

5.2 [P0323] `std::expected`

5.3 Renwick exceptions

5.4 [P0709] Zero-overhead deterministic exceptions: Throwing values

6 Applied papers and issues

7 Retired papers

7.1 [P0829] Freestanding Proposal

7.2 [P1641] Freestanding Library: Rewording the Status Quo

7.3 [P1105] Leaving no room for a lower-level language: A C++ Subset

7.4 [P1212] Modules and freestanding

7.5 [P1372] Giving `atomic_ref` implementers more flexibility by providing customization points for non-lock-free implementation

8 Acknowledgments

References

Informative References

§ 1. Introduction

"Fixing" freestanding is too large of a task for one person. This paper is describing the technical directions that could be taken to make conforming freestanding C++ a useful and widely implemented language. The author does not have the time or resources to perform the due diligence testing and implementation for all of these facilities. This paper is largely a request for collaborators on freestanding related facilities. Please contact ben.craig@gmail.com if you are interested in creating prototype implementations and coauthoring papers on these topics.

§ 2. [\[P0829\]](#) pieces

The following sections are all smaller pieces of the retired [\[P0829\]](#). They are listed in roughly priority order. All of these pieces have some amount of freestanding implementation experience.

§ 2.1. `<cstdlib>`, `<charconv>`, `<cmath>`, `<cinttypes>`, `<cstring>`, `<wchar>`, and `char_traits`

Adding the relevant parts of the above facilities to freestanding requires answering a few questions.

- What do we want to do with floating point in the freestanding library?
- What do we want to do with parts of the C library that could be made freestanding, but aren't freestanding in C?

My recommendation will be that we `=delete` floating point overloads. This way, freestanding implementations with core language floating point support won't have differing runtime library behavior on hosted and freestanding implementations. If the floating point overloads were omitted, then a `float` argument would resolve to an integer overload in freestanding, and a floating point overload in hosted implementations. `<charconv>` floating point overloads are particularly

challenging, as modern implementations of these facilities involve enormous lookup tables that would prove problematic on space constrained implementations.

The other major alternative is to keep floating point operations in freestanding implementations. This would be the "wrong" behavior for kernel users, as floating point usage on many platforms (32-bit x86 Windows, most Linux platforms) will corrupt the floating point state in user mode code.

Adding C functions to C++ freestanding has precedent, notably with `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit`. This is an issue to bring to the newly formed C/C++ liaison study group. `<wchar>` may end up being a casualty to consensus here, as even though it would be a useful facility that fundamentally makes sense in freestanding, it is a costly to port facility (because of assembly) that is rarely used.

§ 2.2. `[diagnostics]`, `<algorithm>`, `<numeric>`, `lock_guard`, `unique_lock`, `<span>`

The above list of facilities is largely a grab-bag, but many of them are more useful if `memcpy` is made available in the `<cstring>` paper.

Very little in `[diagnostics]` is usable in freestanding, but the error code definitions in `<cerrno>` and `<system_error>` are fine.

The details on specifically what to keep for these headers can be extracted from [\[P0829\]](#).

§ 2.3. Partial classes

`array`, `string_view`, `variant`, `optional`, and `bitset` are all classes that are fundamentally compatible with freestanding, but they all have non-critical functions that are not compatible. These are generally functions that throw exceptions, or, in `bitset`'s case, use `std::string`.

[\[P0829\]](#) highlights which functions need to be addressed. Rather than omitting the throwing functions, the functions should be "=deleted", so as to avoid overload resolution differences. This is needed for cases where users derive from standard types and add overloads of standard functions.

`bitset` is more complicated because of the constructor overload set. One of the overloads accepts a `basic_string`, and another accepts `charT *`. Passing a `string_view` to `bitset` seems entirely reasonable. Given the difficulty in making this work, it may be prudent to *not* bring `bitset` to freestanding. It is the author's belief that this would not be a great loss, as `bitset` doesn't actually solve the bit manipulation problems that kernel and embedded developers commonly encounter.

`string_view` requires `char_traits` and `<cstring>`.

§ 2.4. `<random>`

[P0829] describes which `<random>` facilities are a good fit for freestanding. This paper would need to deal with the specification hurdles of `operator<<`. The iostreams `operator<<` overloads for `<random>` types are not listed in the synopsis, as they are permitted to be either hidden friends, or free functions. This makes it editorially more difficult to omit.

§ 2.5. `<chrono>` / `[time]`

Time arithmetic with underlying integer types is entirely reasonable on freestanding platforms. However, `<chrono>` `time_point`s need an underlying Cpp17Clock. Cpp17Clock requires a `now()` function, which requires OS or hardware knowledge that is not appropriate for a freestanding implementation. This paper would need to figure out what should be done with the existing types that satisfy the Cpp17Clock requirements. Note that the in-flight [P2212: Relax Requirements for time_point::clock](#) should be kept in mind when working through the details.

§ 3. Beyond [P0829]

The following are papers that go beyond what [P0829] proposes. These all need implementation experience before being proposed to WG21.

§ 3.1. `constexpr` as `constexpr`

Freestanding has restrictions on the execution environment, but generally doesn't have restrictions on the compilation environment. In principle, everything that can be done at compile time for a hosted target can also be done at compile time for a freestanding target. Having a compile time `std::vector` would be beneficial, even on systems without a heap.

A paper in this area would add blanket wording that ensures everything in the standard library that is `constexpr` is available at compile time for freestanding implementations. If the entity is marked `// freestanding`, then it would be available at runtime as well. This would generally be implemented (and possibly specified) as making the function optionally `constexpr` on freestanding

implementations. The freestanding implementation would either provide a `constexpr` version of the facility, or a `consteval` version if the target can support it.

This paper would need to audit the standard library's `constexpr` facilities and audit the `constexpr` papers in flight. The paper would need to resolve complications of split overload sets, if any exist.

The paper would need to evaluate semantic changes that could occur when porting freestanding code to hosted, particularly for classes that are `constexpr` but not freestanding (like `std::vector` and `std::string`). `consteval` objects can be destroyed at different times than `constexpr` objects.

A `consteval std::string` may be part of a solution to excluding `std::bitset` functions that take a `std::string` at runtime.

It may also make sense to split this into multiple papers: one that covers all `constexpr` non-member functions, and another that covers all `constexpr` classes.

§ 3.2. Startup and termination control

§ 3.2.1. Replaceable `std::terminate` `{#replaceable_terminate}`

By default, `std::terminate` delegates the work of termination to `std::abort`. `std::abort` will then use OS facilities to terminate the process. On freestanding platforms, there's a good chance that the C library `std::abort` won't link, because it is attempting to use OS facilities that don't exist. In addition, there are often additional or customized actions that need to be performed on embedded platforms.

C++ already has a way to provide additional or customized actions for `std::terminate`, and that is by changing the terminate handler at runtime. The problem with this approach is that runtime is too late. `std::abort` will be odr-used at build time, no matter how early in the runtime process the user switches the termination handler. The user may end up linking in an otherwise unused atomics support library for `set_terminate` to use so that it won't cause a data race.

A paper addressing this would make `std::terminate` a replaceable function, similar to `::operator new`. Input would be needed from the ABI review group, as it is unclear whether making `std::terminate` replaceable would be an ABI break or not.

A further step could be taken to make `std::terminate` optional on freestanding implementations, similar to how `::operator new` was made optional in [P2013]. This would be controversial, as it would effectively make exceptions, `dynamic_cast`, and other features (`[except.terminate]`) optional as well. Note that there is usage experience for not having `std::terminate` or its associated features ([P1105]).

If `std::terminate` were made a build-time replaceable function, then `get_terminate` and `set_terminate` would no longer need to be freestanding.

§ 3.2.2. Link-time arrays {#link_time_arrays}

C++ linkers have the ability to take symbols from multiple translation units and merge those symbols into a single array. This facility is used as an implementation strategy for global constructors, thread local storage areas, and exception handling tables. Each translation unit's object file advertises some portion of the array. The linker then either concatenates the arrays together in the final binary, or does some other kind of merging operation.

It would be useful to expose this facility directly to the programmer. Unit test frameworks usually have macros that create global objects that register a unit test with a singleton during program startup. All the registrants are known at link time, yet we end up paying a runtime cost for the registration. LLVM internals have optimization passes that are also registered via global object constructors.

Here's some expositional, strawman syntax to further convey the ideas suggested above.

```
// test_framework.h
using test_func = bool (*);
// declare link-time array
extern const test_func g_tests[register];

// always_pass.cpp
#include "test_framework.h"
bool always_pass() {return true;}
// add items to link-time array
const test_func g_tests[register] = {always_pass};

// always_fail.cpp
#include "test_framework.h"
bool fail1() {return false;}
bool fail2() {return false;}
// add items to link-time array
const test_func g_tests[register] = {fail1, fail2};

// main.cpp
#include "test_framework.h"
int main() {
    // g_tests contains {always_pass, fail1, fail2},
    // though not necessarily in that order
```

```

    for(test_func f : g_tests) {
        if(!f()) return 1;
    }
    return 0;
}

```

The use of `register` in the above code is strawman syntax. The author of this future paper needs to figure out how to expose the end of the array. This could be done with a non-constant-expression `sizeof`. It could be done with a "magic" type, perhaps `std::span`. It could be done by exposing an end iterator / pointer. There are likely other options as well.

Link-time arrays are a useful primitive, as evidenced by their existing use for other language features. Exposing them to programmers would allow for opt-in fixes to current problems. Imagine having a link-time array that contains a "priority" value alongside an initialization function. The consumer of the array could then sort the array (or a copy of it) by priority, then run the initialization functions. This would portably solve some users issues with static initialization ordering by giving users the power to express the desired ordering.

Link-time arrays could also provide a way to call "global constructors" in a dynamic library outside of various operating system locks.

Link-time arrays could give users enough control to manually destroy and recreate an open set of global resources.

This feature should probably be restricted to global arrays. It may also make sense for an initial version to be restricted to some category of simple types (trivial types? literal types?).

It is unclear to me how this feature could expose heterogenous "arrays" like those needed for `thread_local` storage initialization. Getting the alignment right on individual elements seems particularly challenging.

§ 3.2.3. Exposing existing global constructor link-time arrays `{#expose_global_ctors}`

Link-time arrays on their own are useful for freestanding, but not uniquely so. The reason they get mentioned in this freestanding roadmap is that implementations could then portably expose existing facilities, like the global constructor list. Kernel and embedded users could then portably control when the global constructors are called.

An author of such a paper needs to be sure to keep the destruction side of things in mind as well.

§ 3.2.4. `thread_local` customization points `{#thread_local_customization}`

`thread_local` currently requires the compiler, linker, and operating system to collaborate on a `thread_local` protocol. When the OS creates a thread, it will often want to know how much storage to set aside for `thread_local` variables. It will want to know what the initial contents of that storage should be. Currently, that information is not made portably available to users authoring their own OS.

Once the per-thread storage is initialized, the compiler needs to know how to find the storage when accessing a variable. Perhaps an extension point that exposes a numeric thread ID can be the bridge between the compiler runtime and user code.

This potential paper has many unknowns. There are many challenges regarding ABI compatibility and existing implementations. It may also make sense to make `thread_local` support on freestanding implementations optional.

SG1 in the San Diego 2018 meeting provided this feedback on [\[P1105\]](#):

Conforming freestanding implementations could make `thread_local` ill-formed

SF F N A SA

3 12 5 0 0

§ 3.3. Error handling

Error handling as a whole is the area in most need of work in freestanding. There aren't currently any good, standardized error handling facilities for freestanding platforms.

The following facilities could help make [Renwick exceptions](#) or [\[P0709\]](#) exceptions more useful on freestanding platforms, while still satisfying the underlying use cases.

§ 3.3.1. Destructor call reason `{#dtor_call_reason}`

[P0052: Generic Scope Guard and RAII Wrapper for the Standard Library](#) proposes `scope_fail` and `scope_success` (among other facilities). These facilities need to know whether a destructor is being called as part of unwinding, or as part of normal execution. The reference implementation relies on `std::uncaught_exceptions`, which typically relies on some form of thread-local storage.

In practice, thread-local storage is not widely available on freestanding platforms. In addition, thread-local storage has concurrency issues with coroutines that migrate threads, as well as [\[P0876\]](#) fibers. It would be nice to be able to implement `scope_fail`, `scope_success`, and similar facilities without the need to rely on thread-local storage.

One way is to have an alternate signature for destructors that allows the user to discriminate between the two cases.

```
struct Strawman {
    ~Strawman(bool success) {
        if(success) {
            success_action();
        } else {
            fail_action();
        }
    }
};
```

We only need to allow one destructor per class. A class with boolean destructor would only be able to be used as a non-member, a member of a class with a boolean destructor, or as a subobject of a class with a boolean destructor. Classes with void destructors could be members of classes with boolean destructors. More definition and exploration is needed around things like inheritance and virtual destructors.

This general approach would likely make classes like `scope_success` and `scope_fail` faster and smaller, as the happy path doesn't need to capture any state to compare later. It removes the need for thread-local storage for this use case.

§ 3.3.2. Non-local exception objects {#non_local_exception}

`std::uncaught_exceptions` isn't the only way in which the exception handling mechanism uses thread-local storage. `std::current_exception` and `throw`; also use thread-local storage. If we could directly capture a `std::exception_ptr`, or something similar to one, then we could avoid thread-local storage.

```
// strawman syntax only
int may_throw();
std::expected<int, std::ex_ptr2> f() {
    try {
        return may_throw();
    } catch(std::ex_ptr2 e : std::exception &orig) {
        // catches everything derived from std::exception, provides
        // an ex_ptr2 for capturing and rethrowing purposes
        std::cout<<orig.what();
        return std::unexpected(e);
    }
}
```

```

    } catch(std::ex_ptr2 e : ...) {
        // catches everything, provides an ex_ptr2 for capturing and
        // rethrowing purposes
        return std::unexpected(e);
    }
}

```

Such a facility could be used to implement ["Lippincott" functions](#), or to transport an exception across threads. It may be possible to reuse the current `std::exception_ptr` for this facility, but reuse here shouldn't be considered a necessary design constraint. A new type may be able to do deep, stack based copies of exception objects, potentially avoiding using the heap entirely (offer not valid in Itanium ABI jurisdictions).

There is likely a lot of overlap between this suggestion, and [P1066: How to catch an `exception\_ptr` without even `try`-ing](#).

§ 3.3.3. `noexcept` blocks `{#noexcept_blocks}`

One of the ways to combat exception overhead is by accurately marking large numbers of functions as `noexcept`. This only reduces overhead when the `noexcept` function only calls other functions that the implementation can prove don't throw. Marking lots of functions as `noexcept` adds a fair amount of syntactic noise though. `noexcept` blocks could reduce that noise.

```

noexcept {
    void does_not_throw();
    void also_does_not_throw();
    void may_throw() noexcept(true);
}

```

One of the major challenges in accurate `noexcept` annotation is dealing with functions from C libraries. `extern "C"` functions are allowed to throw, but most don't. Very few C functions are marked as `noexcept`. `noexcept` blocks are one way to deal with that.

```

noexcept {
    #include "some_c_header.h"
}

```

This is likely to lead to (usually harmless) ODR violations.

§ 3.3.4. Trim exception header {#trim_exception_header}

If and when destructor call reasons and non-local exception objects get standardized, we could remove freestanding-hostile functionality from freestanding. In particular, we could stop requiring `uncaught_exceptions()`, `current_exception()`, and `throw;` from being required on freestanding implementations. This may be sufficient for something like [Renwick exceptions](#) or [\[P0709\]](#) to be successful without requiring thread-local storage.

§ 3.4. No known path forward

The following topics are problem areas for freestanding, but the author doesn't have any great ideas on how to address them in WG21.

§ 3.4.1. Locked atomics {#locked_atomics}

Operating systems and embedded devices often need to write interrupt handling code. The rules for interrupt-safe code are very similar to the rules for signal-safe code. Locked atomics / non-lock-free atomics are a portability trap in these environments.

[\[P1105\]](#) proposed making locked atomics ill-formed on freestanding platforms.

SG1 in the San Diego 2018 meeting provided this feedback on [\[P1105\]](#):

Conforming freestanding implementations could omit lock-free[sic] atomics

SF F N A SA

2 6 4 6 2

[--Note The wording of the poll seems to have been minuted slightly wrong. Omitting non-lock-free atomics was the intent. --end note --]

§ 3.4.2. Thread-safe statics {#thread_safe_statics}

Thread-safe statics require locking support. Similar to locked atomics, there is no one right choice.

[\[P1105\]](#) Proposed making thread-safe statics ill-formed on freestanding platforms.

SG1 in the San Diego 2018 meeting provided this feedback on [\[P1105\]](#):

Conforming freestanding implementations could omit thread-safe statics

SF F N A SA

0 6 8 4 2

§ 3.4.3. RTTI {#rtti}

The non-throwing parts of RTTI are implementable on most freestanding platforms. The issue is that it is very hard to optimize away unused `type_infos`.

[P1105] recommends making `typeid(expr)` and `dynamic_cast` ill-formed on freestanding platforms. It seems unlikely that such a direction will gain consensus in WG21.

§ 3.4.4. Floating point {#floating_point}

Unprotected use of floating point operations in kernel environments will corrupt the user mode state on many operating systems. This is true even when the operations are just loads and stores. Some operating systems provide a way to explicitly save and restore the floating point state, so that floating point can be used in a given region.

Embedded processors often do not have dedicated floating point units. On these platforms, floating point is emulated in software. This floating point emulation is often very large in terms of code size.

Ideally, there would be a portable way to discourage floating point use in these environments, without making it impossible.

[P1105] recommended making the floating point types ill-formed in environments where floating point isn't supported. This approach would be difficult to get consensus on. It would also make it much more difficult to get consistent overload set behavior when moving code between floating point and hosted environments.

§ 4. Work in progress

§ 4.1. [P2013] Freestanding Language: Optional `::operator new`

P2013R1 is awaiting further EWG review. EWG reviewed P2013R0 favorably in the 2020 Prague meeting, and requested wording. Wording is present and ready for review in P2013R1.

§ 4.2. [P1642] Freestanding Library: Easy [utilities], [ranges], and [iterators]

P1642R4 combines the wording in P1641R3 and the library additions from P1642R3. P1642R4 is awaiting further LEWG review.

The non-feature test macro parts of P1641R3 and P1642R3 were reviewed favorably over LEWG telecon. LEWG requested that those parts be combined into a single paper, and that LEWG not spend future time discussing the contents of the library additions.

§ 4.3. [\[P2198\] Freestanding Feature-Test Macros and Implementation-Defined Extensions](#)

P2198R0 has taken all the feature-test macro parts from P1642R3 and P1641R3, and consolidated them here. P2198R0 needs to be reviewed by SG10, as it is doing something more involved than just including a feature test macro.

§ 5. Related work in progress

§ 5.1. [\[P0581\] Standard Library Modules](#)

The way we partition the standard library into modules has the potential to change how freestanding is advertised and messaged significantly. The partitioning could also place new constraints on freestanding.

§ 5.2. [\[P0323\] `std::expected`](#)

Error handling is currently the largest sore point in freestanding, and `std::expected` is one proven approach that can address many error handling needs.

§ 5.3. [Renwick exceptions](#)

This paper describes an implementation method of exceptions that could resolve many long standing implementability and performance issues with exceptions in freestanding environments. This is currently the best hope that C++ has for a one-size-fits-most error handling mechanism.

Most of the work in this paper is beyond the authority of WG21. The paper asks for ABI changes, and for implementers to make different trade-offs than they have traditionally made. The paper also asks the freestanding community to use a language feature that they have traditionally shunned.

§ 5.4. [\[P0709\]](#) Zero-overhead deterministic exceptions: Throwing values

This paper describes an additional form of exception handling that can coexist with today's C++ exceptions. This is currently the best hope that C++ has for exceptions where the user can make function-specific performance trade-offs.

If this paper gets accepted, then there will likely be a need for many follow-on papers to parameterize the error handling for many C++ facilities. Users would want to be able to provide an allocator to `std::vector` that would make that instantiation use P0709 exceptions. Users would want to be able to make coroutine promise types report allocation failures through P0709 exceptions. Users may even want a variety of `dynamic_cast` that uses P0709 exceptions.

§ 6. Applied papers and issues

- [\[LWG3148\]](#) `<concepts>` should be freestanding
- [\[P1855\]](#) Make `<compare>` freestanding

§ 7. Retired papers

§ 7.1. [\[P0829\]](#) Freestanding Proposal

P0829 was too large to be effectively reviewed, so it has been split up. [\[P1641\]](#), [\[P1642\]](#), and [\[P2198\]](#) are the current successors to P0829, but more are needed. See [§ 2 \[P0829\] pieces](#) for some of the other papers that still need to be authored.

§ 7.2. [\[P1641\]](#) Freestanding Library: Rewording the Status Quo

Most of the wording aspects of this paper have moved to P1642, and the feature test macro parts have moved to P2198.

§ 7.3. [\[P1105\]](#) Leaving no room for a lower-level language: A C++ Subset

P1105 was meant to describe a direction. Follow-on papers were meant to act on that direction. P2013 is one such paper.

When SG1 reviewed this paper in the 2018 San Diego meeting, they reacted favorably to the `thread_local` parts, and unfavorably to the locked atomics and thread safe static parts.

§ 7.4. [P1212] Modules and freestanding

This paper was a reaction to [P0581]. One of the questions this paper asked was to put the language support facilities that require an operating system in a different module from the freestanding facilities. EWG was opposed to this direction in the San Diego 2018 meeting. The author's position regarding freestanding and standard library modules has changed significantly since the San Diego 2018 meeting.

§ 7.5. [P1372] Giving `atomic_ref` implementers more flexibility by providing customization points for non-lock-free implementation

This paper was not received favorably by SG1 in San Diego's 2018 meeting.

§ 8. Acknowledgments

Thank you to Brandon Streiff for reviewing this paper and providing additional use cases for link-time arrays.

§ References

§ Informative References

[Guillemot]

Nicolas Guillemot. Using a Lippincott Function for Centralized Exception Handling. URL: <http://cppsecrets.blogspot.com/2013/12/using-lippincott-function-for.html>

[LWG3148]

Casey Carter. `<concepts>` should be freestanding. URL: <https://wg21.link/LWG3148>

[P0052]

Peter Sommerlad; Andrew L. Sandoval. Generic Scope Guard and RAII Wrapper for the Standard Library. URL: <https://wg21.link/P0052r10>

[P0323]

Vicente Botet; JF Bastien. `std::expected`. URL: <https://wg21.link/P0323>

[P0581]

Marshall Clow; et al. [Standard Library Modules](http://wg21.link/P0581). URL: <http://wg21.link/P0581>

[P0709]

Herb Sutter. [Zero-overhead deterministic exceptions: Throw values](http://wg21.link/P0709). URL: <http://wg21.link/P0709>

[P0829]

Ben Craig. [Freestanding Proposal](https://wg21.link/P0829). URL: <https://wg21.link/P0829>

[P0876]

Oliver Kowalke; Nat Goodspeed. [fiber_context- fibers without scheduler](https://wg21.link/P0876R10). URL: <https://wg21.link/P0876R10>

[P1066]

Mathias Stearn. [How to catch an exception_ptr without even try-ing](http://wg21.link/P1066R1). URL: <http://wg21.link/P1066R1>

[P1105]

Ben Craig; Ben Saks. [Leaving no room for a lower-level language: A C++ Subset](https://wg21.link/P1105R1). URL: <https://wg21.link/P1105R1>

[P1212]

Ben Craig. [Modules and Freestanding](https://wg21.link/P1212R0). URL: <https://wg21.link/P1212R0>

[P1372]

Daisy Hollman. [Giving `atomic\\_ref` implementers more flexibility by providing customization points for non-lock-free implementation](https://wg21.link/P1372R0). URL: <https://wg21.link/P1372R0>

[P1641]

Ben Craig. [Freestanding Library: Rewording the Status Quo](https://wg21.link/P1641R3). URL: <https://wg21.link/P1641R3>

[P1642]

Ben Craig. [Freestanding Library: Easy \[utilities\], \[ranges\], and \[iterators\]](https://wg21.link/P1642R4). URL: <https://wg21.link/P1642R4>

[P1855]

Ben Craig. [Make <compare> freestanding](https://wg21.link/P1855R0). URL: <https://wg21.link/P1855R0>

[P2013]

Ben Craig. [Freestanding Language: Optional `::operator new`](https://wg21.link/P2013R3). URL: <https://wg21.link/P2013R3>

[P2198]

Ben Craig. [Freestanding Feature-Test Macros and Implementation-Defined Extensions](https://wg21.link/P2198R1). URL: <https://wg21.link/P2198R1>

[P2212]

Alexey Dmitriev; Howard Hinnant. [Relax Requirements for time_point::clock](https://wg21.link/p2212r2). URL: <https://wg21.link/p2212r2>

[RenwickLowCost]

James Renwick; Tom Spink; Bjoern Franke. [Low-cost Deterministic C++ Exceptions for Embedded Systems](https://www.research.ed.ac.uk/portal/en/publications/lowcost-deterministic-c-exceptions-for-embedded-systems(2cfc59d5-fa95-45e0-83b2-46e51098cf1f).html). URL: [https://www.research.ed.ac.uk/portal/en/publications/lowcost-deterministic-c-exceptions-for-embedded-systems\(2cfc59d5-fa95-45e0-83b2-46e51098cf1f\).html](https://www.research.ed.ac.uk/portal/en/publications/lowcost-deterministic-c-exceptions-for-embedded-systems(2cfc59d5-fa95-45e0-83b2-46e51098cf1f).html)